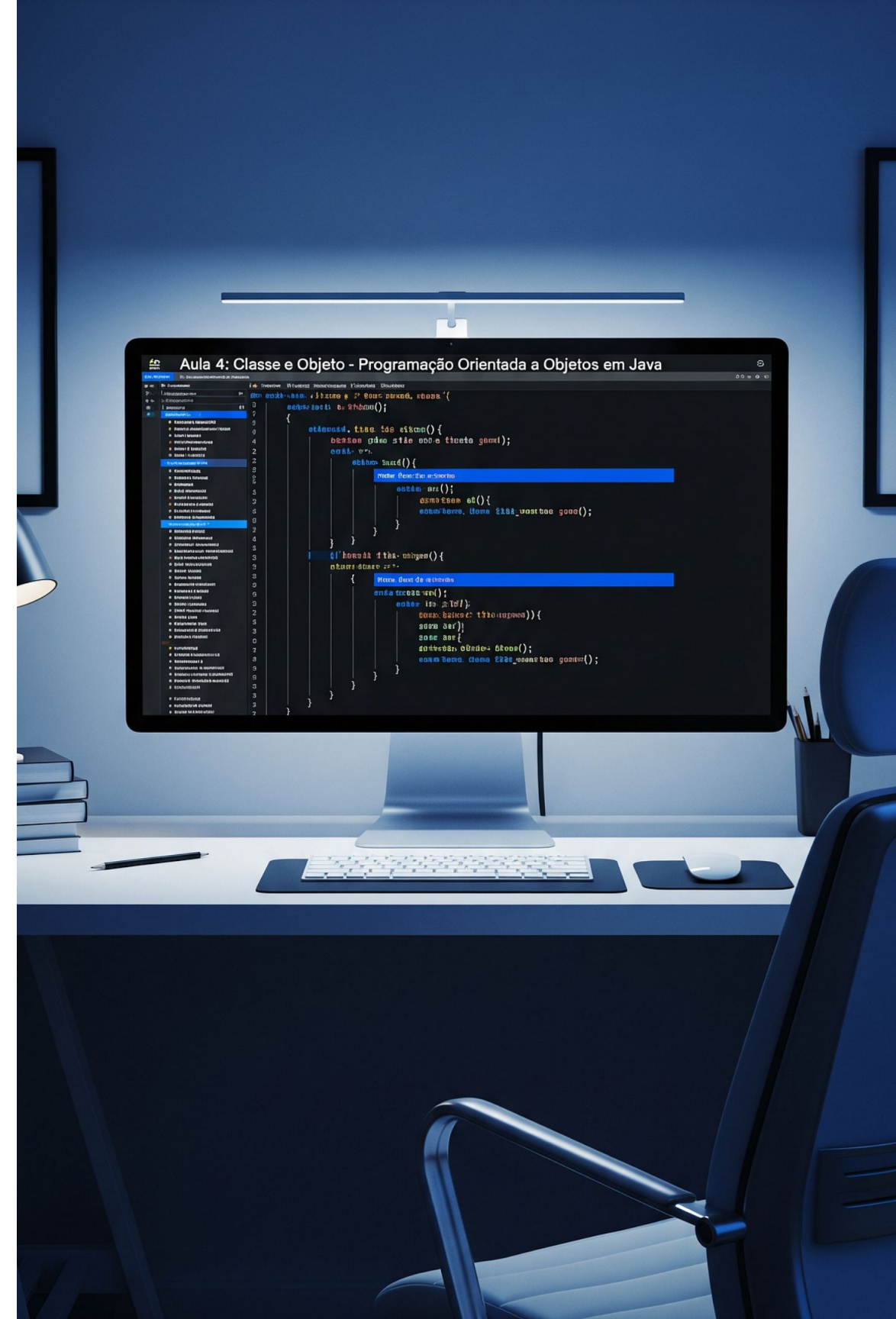


POO EM

JAVA

Aula 4: Classe e Objeto

Programação Orientada a Objetos em Java — Fundamentos essenciais para modelar o mundo real em código.



Objetivos de Aprendizagem

Ao final desta aula, você será capaz de compreender e aplicar os conceitos fundamentais de Classe e Objeto em Java. Esses são os blocos de construção de qualquer sistema orientado a objetos.

1

Definir Classe e Objeto

Compreender a diferença entre molde (classe) e instância (objeto).

2

Escrever Classes em Java

Declarar classes com atributos, métodos e modificadores de acesso.

3

Instanciar Objetos

Usar o operador `new` para criar e manipular objetos em memória.

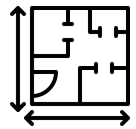
4

Acessar Membros

Utilizar o operador ponto (`.`) para acessar atributos e chamar métodos.

O que é uma Classe?

Uma **classe** é um molde, um template que define a estrutura e o comportamento de um conjunto de objetos. Ela não existe fisicamente na memória durante a execução, ela descreve *como* os objetos desse tipo serão formados.



Planta de uma Casa

A planta não é a casa, e sim o projeto.

Com ela, você pode construir muitas casas idênticas.

A classe é exatamente isso: o projeto do objeto.



Forma de Bolo

Uma forma de bolo define o formato de todos os bolos feitos com ela, mas ela mesma não é comestível. A classe define, o objeto é o resultado concreto.



Definição Formal

Em Java, uma classe é uma unidade de código que agrupa **atributos** (dados) e **métodos** (comportamentos) relacionados a um mesmo conceito.

Anatomia de uma Classe

1 Atributos

Variáveis que armazenam o **estado** do objeto. Representam as características: `nome`, `idade`, `velocidade`. Cada objeto tem seus próprios valores.

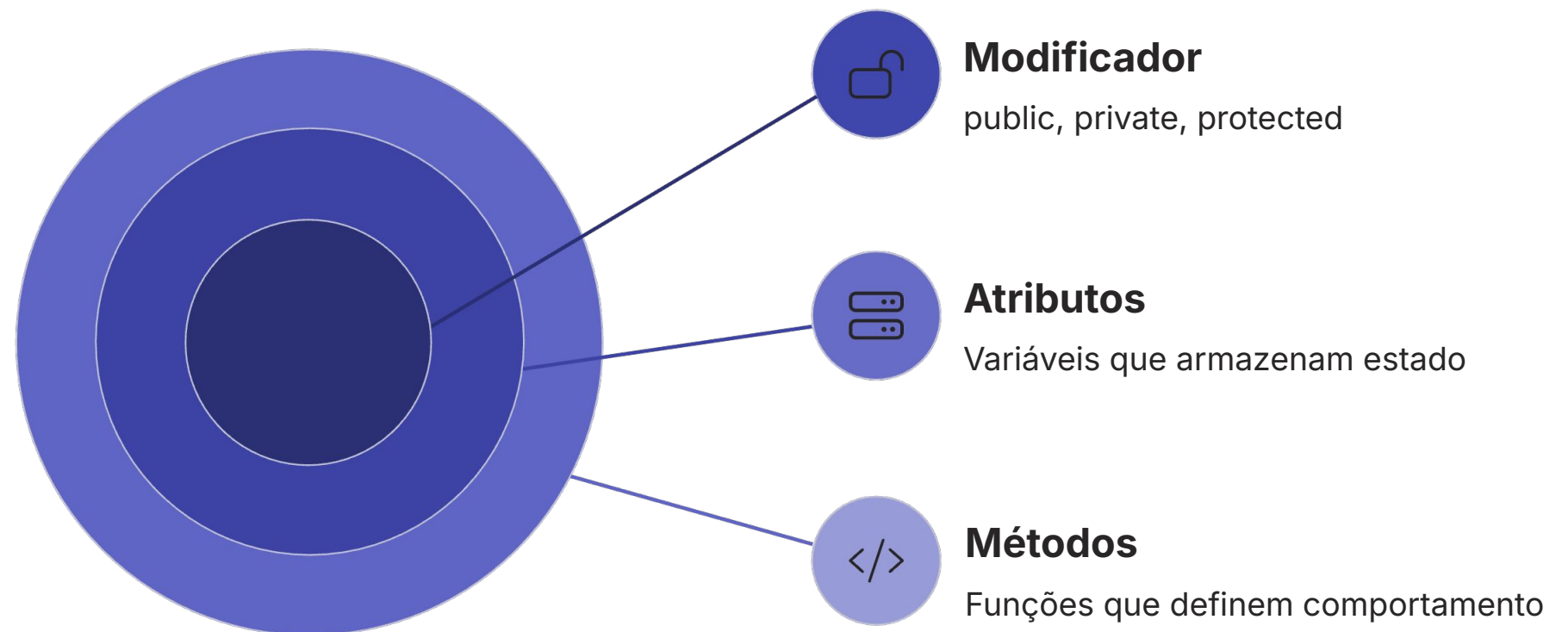
2 Métodos

Funções que definem o **comportamento** do objeto. Representam as ações: `acelerar()`, `calcularNota()`. Definem o que o objeto sabe fazer.

3 Modificadores de Acesso

Controlam a visibilidade dos membros: `public` (visível a todos), `private` (apenas na classe), `protected` (classe e subclasses).

Toda classe Java é composta por três elementos fundamentais que definem sua estrutura interna. Compreender cada parte é essencial para escrever código orientado a objetos correto e organizado.



Sintaxe de Classe em Java

```
public class Carro {  
  
    // — Atributos (estado do objeto) —————  
    private String marca;  
    private String modelo;  
    private int    ano;  
    private double velocidadeAtual;  
  
    // — Métodos (comportamento do objeto) —————  
    public void acelerar(double incremento) {  
        velocidadeAtual = velocidadeAtual + incremento;  
    }  
  
    public void frear(double decremento) {  
        velocidadeAtual = velocidadeAtual - decremento;  
    }  
  
    public double getVelocidadeAtual() {  
        return velocidadeAtual;  
    }  
}
```




O que é um Objeto?

Objeto é um exemplo real criado a partir de uma classe. Se a classe é o modelo, o objeto é aquilo que foi criado com base nele. Cada objeto tem seus próprios valores e existe separadamente na memória do programa.

Entidade Concreta

O objeto ocupa espaço real na memória (heap). Ele tem **identidade** (endereço único), **estado** (valores dos atributos) e **comportamento** (métodos que pode executar).

Instância de Classe

Vários objetos podem ser criados a partir da mesma classe. Um Carro pode gerar `meuFiat`, `seuGol` e `carroDoJoao` — cada um com sua própria cor, velocidade e ano.

Classe vs. Objeto

Esta é a distinção mais importante da POO. Entender a diferença entre a definição abstrata (classe) e a entidade concreta em memória (objeto) é fundamental para raciocinar sobre o seu código.

◆ CLASSE

- Definição abstrata / molde
- Existe no código-fonte (.java)
- Criada uma única vez pelo programador
- Não ocupa espaço no heap

Exemplo: `class Carro { ... }`

1

Classe criada

Uma única definição no código-fonte

● OBJETO

- Entidade concreta / instância
- Existe na memória em tempo de execução
- Pode existir em múltiplos exemplares
- Ocupa espaço real no heap da JVM

Exemplo: `new Carro()`

∞

Objetos possíveis

Ilimitadas instâncias em tempo de execução

CÓDIGO

JAVA

Criação de Objetos em Java

Para criar um objeto, usamos o operador `new` seguido do **construtor** da classe. O Java aloca memória no heap, inicializa o objeto e retorna uma **referência** que armazenamos em uma variável.

```
// 1. Declaração da variável de referência
```

```
Carro meuCarro;
```

```
// 2. Instanciação com o operador new
```

```
meuCarro = new Carro();
```

```
// — Forma compacta (mais comum) —————
```

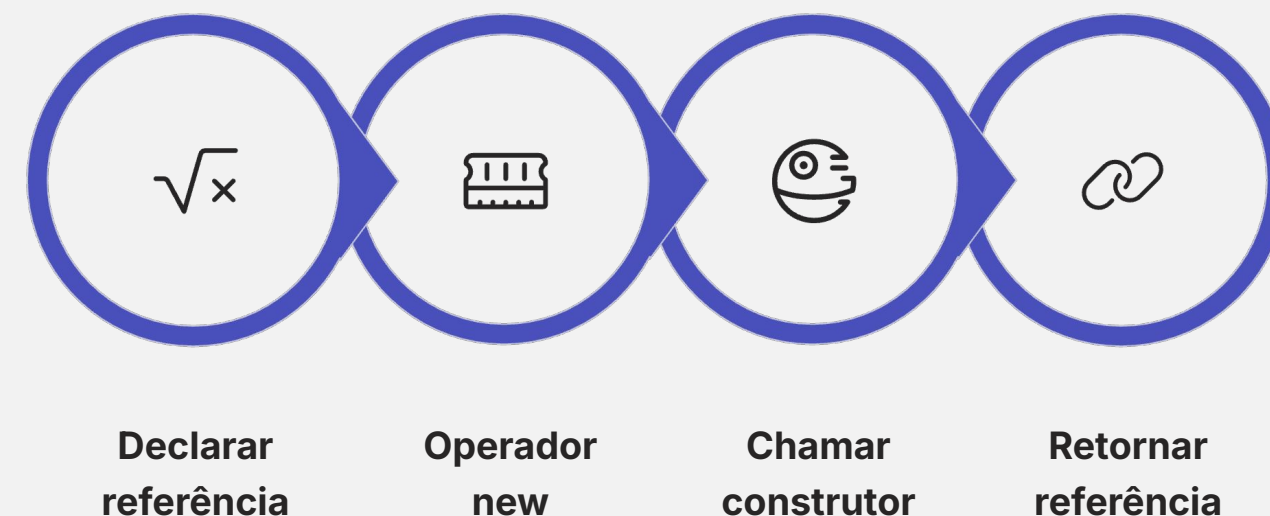
```
Carro outroCarro = new Carro();
```

```
// — Múltiplos objetos da mesma classe —————
```

```
Carro fiat    = new Carro();
```

```
Carro toyota  = new Carro();
```

```
Carro honda   = new Carro();
```



Atributos — Variáveis de Instância

Atributos (também chamados de **variáveis de instância**) armazenam o estado individual de cada objeto. Cada objeto criado tem sua própria cópia dos atributos, com valores independentes dos demais objetos.

```
public class Aluno {  
  
    // Tipos primitivos  
    private int    matricula;  
    private double mediaFinal;  
    private boolean ativo;  
  
    // Tipos referência  
    private String nome;  
    private String curso;  
  
}
```

Valores Padrão

Se não inicializados, atributos recebem valores padrão: `int` → `0`, `double` → `0.0`, `boolean` → `false`, `String` → `null`.

Independência de Estado

Se você tem dois objetos `Aluno`, alterar o `nome` de um não afeta o outro. Cada objeto é autônomo em seu estado.

Métodos — Comportamento do Objeto

Métodos definem o que um objeto *sabe fazer*. Eles podem receber parâmetros, processar dados internos e retornar valores. A **assinatura** de um método é composta por nome, parâmetros e tipo de retorno.

```
public class Aluno {  
    private String nome;  
    private double nota1, nota2;  
  
    // Método sem retorno (void) e com  
    parâmetro  
    public void setNome(String novoNome) {  
        nome = novoNome;  
    }  
  
    // Método com retorno (double) sem  
    parâmetro  
    public double calcularMedia() {  
        return (nota1 + nota2) / 2.0;  
    }  
}
```

```
// Método com retorno (String) sem  
parâmetro  
    public String getSituacao() {  
        if (calcularMedia() >= 6.0) {  
            return "Aprovado";  
        } else {  
            return "Reprovado";  
        }  
    }  
}
```

Acessando Membros do

Objeto O operador ponto (.) é a forma de interagir com um objeto em Java. Por meio dele, acessamos seus atributos (quando public) e chamamos seus métodos. É a sintaxe universal da POO.

```
// Criando o objeto
Aluno aluno1 = new Aluno();

// Chamando métodos para definir o estado
aluno1.setNome("Maria Silva");
aluno1.setNota1(8.5);
aluno1.setNota2(7.0);
```

```
// Chamando método que retorna valor
double media = aluno1.calcularMedia();
System.out.println("Média: " + media);    //
7.75

// Chamando método que usa lógica interna
String sit = aluno1.getSituacao();
System.out.println("Situação: " + sit);    //
Aprovado
```

1

Referência

`aluno1` aponta para o objeto no heap

2

Operador .

Navega até o membro desejado

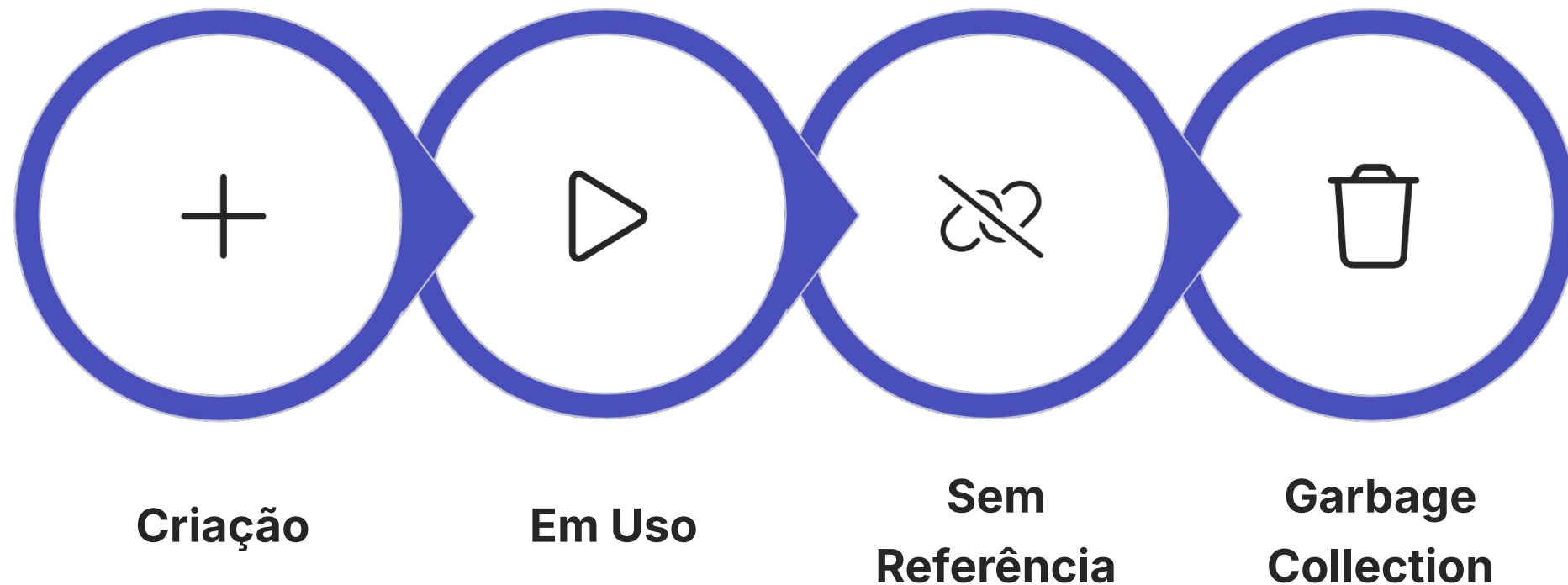
3

Membro

Atributo ou método do objeto é acessado

Ciclo de Vida do Objeto

Todo objeto em Java passa por fases bem definidas: da criação até a destruição automática pelo **Garbage Collector** da JVM. Entender esse ciclo ajuda a evitar erros como `NullPointerException` e vazamentos de memória.



Referência Nula

Quando você declara `Carro c;` sem o `new`, a variável `c` vale `null`. Tentar acessar `c.marca` causa `NullPointerException`.

Garbage Collector

Quando nenhuma variável referencia um objeto, a JVM o marca para coleta. Você não precisa destruir objetos manualmente em Java — diferente de C/C++.

Exemplo Prático: Classe Aluno

```
// — Definição da Classe —————  
public class Aluno {  
    private String nome;  
    private int    matricula;  
    private double nota1, nota2;  
  
    public void setNome(String n)      { nome = n; }  
    public void setMatricula(int m)    { matricula = m; }  
    public void setNota1(double n)     { nota1 = n; }  
    public void setNota2(double n)     { nota2 = n; }  
  
    public String getNome()            { return nome; }  
    public double calcularMedia()       { return (nota1 + nota2) / 2.0; }  
  
    public void exibirResumo() {  
        System.out.println("Aluno: " + nome);  
        System.out.println("Média: " + calcularMedia());  
        System.out.println("Status: " + (calcularMedia() >= 6 ? "Aprovado" : "Reprovado"));  
    }  
}
```

Exemplo Prático: Classe Aluno

```
// — Uso da Classe (método main) —————  
public class Main {  
    public static void main(String[] args) {  
        Aluno a1 = new Aluno();           // criando objeto  
        a1.setNome("Ana Costa");  
        a1.setMatricula(20240001);  
        a1.setNota1(9.0);  
        a1.setNota2(8.5);  
        a1.exibirResumo();                 // chamando método  
    }  
}
```

Boas Práticas

Adotar boas práticas desde o início é o que diferencia um código amador de um profissional. Em Java, convenções de nomenclatura e princípios de design são amplamente seguidos pela comunidade.

UpperCamelCase para Classes

Nomes de classes sempre começam com letra maiúscula: `Aluno`, `ContaBancaria`, `CarroEsportivo`.
Nunca use `aluno` ou `carro_esportivo`.

Responsabilidade Única

Cada classe deve ter uma única responsabilidade bem definida. Uma classe `Aluno` não deve calcular folha de pagamento — isso seria responsabilidade de outra classe.

lowerCamelCase para Métodos e Atributos

Métodos e atributos iniciam com minúscula: `calcularMedia()`, `nomeCompleto`. São descritivos e indicam claramente seu propósito.

Atributos `private` + Getters/Setters

Declare atributos como `private` e forneça acesso controlado via métodos `getX()` e `setX()`. Este é o princípio básico do encapsulamento.

Erros Comuns — Evite Essas Armadilhas

Iniciantes em POO cometem alguns erros recorrentes. Conhecê-los com antecedência acelera muito o aprendizado e evita horas de depuração frustrante.

❌ NullPointerException

Causa: Usar uma referência que ainda não aponta para nenhum objeto.

```
Aluno a;           // a = null
a.getNome();       //
🔥 NullPointerException!
```

Solução: Sempre use `new` antes de acessar membros: `Aluno a = new Aluno();`

❌ Confundir Classe com Objeto

Erro conceitual: Pensar que a classe é o objeto ou que modificar um objeto afeta a classe.

Lembre-se: A classe é o molde imutável; cada objeto tem seu próprio estado independente.

❌ Esquecer o Operador `new`

Causa: Declarar a variável sem instanciar o objeto.

```
Carro c;
// apenas referência (null)
c = Carro();
// Erro de compilação!
c = new Carro();
// Correto
```

Exercícios Práticos

Coloque em prática tudo o que foi aprendido nesta aula. A melhor forma de fixar conceitos de POO é escrever código — não apenas ler!

01

Exercício 1 — Classe Livro

Crie uma classe `Livro` com atributos `titulo`, `autor`, `numeroPaginas` e um método `exibirInfo()` que imprime os dados.

02

Exercício 2 — Classe ContaBancaria

Crie uma classe com atributos `titular` e `saldo`. Implemente métodos `depositar(double)`, `sacar(double)` e `getSaldo()`.

03

Exercício 3 — Múltiplos Objetos

No `main`, crie 3 objetos da sua classe `Livro`, defina valores diferentes para cada um e exiba todas as informações.



Próxima Aula: Construtores

Na **Aula 5** vamos aprofundar a criação de objetos com **construtores personalizados**: como inicializar atributos no momento da criação, sobrecarga de construtores e o uso da palavra-chave `this`.

Resumo da Aula 4

- Classe = molde / template
- Objeto = instância concreta

Operador `new` instancia

Operador `.` acessa membros